VIDYAVARDHAKA COLLEGE OF ENGINEERING

Autonomous institution affiliated to

Visvesvaraya Technological University, Belagavi

PROJECT SYNOPSIS

Cybersecurity LLM Agents: Automated Threat Analysis Telemetry Assessment and Red-Team Orchestration using Conversational Agents

Vinyas CS	**4VV23CS244**
Yashith Raj MB	**4VV23CS250**
Vishwas CS	**4VV23CS246**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA, New Delhi

VIDYAVARDHAKA COLLEGE OF ENGINEERING

Contents

	Contents	Page
1	Title of the Project	03
2	Abstract	04
3	Introduction and Background	05
4	Problem Statement	06
5	Objectives of the Project	07
6	Proposed Methodology	08
7	Technologies / Tools to be Used	11
8	Expected Outcomes and Applications	12
9	Conclusion	13
10	References	15

1.PROJECT TITLE:

Cybersecurity LLM Agents: Automated Threat Analysis
Telemetry Assessment
and
Red-Team Orchestration using Conversational Agents

2.Abstract

This project proposes and implements a modular framework of conversational agents that leverage Large Language Models (LLMs) to automate common cybersecurity tasks, including document summarization, telemetry-gap detection, sandboxed command execution, and orchestration with an adversary emulation platform (MITRE CALDERA). The system consists of an agent layer (conversational LLM agents) and a deterministic tool layer (web and PDF parsers, shell execution wrappers, Caldera API helpers). The coordinator agent decomposes high-level objectives into tool-enabled subtasks while other agents perform text analysis, internet ingestion, and controlled command execution in a sandboxed workspace. The methodology emphasizes safety: sandboxing, explicit operator approval for destructive actions, credential isolation, and token-aware context management to avoid prompt overflow. Evaluation will measure (1) accuracy and usefulness of text summarization, (2) precision and recall of telemetry-gap detection against a labeled baseline, and (3) operational productivity gains (time-to-triage) in demo scenarios.

Deliverables include a documented codebase, reproducible Jupyter notebooks demonstrating representative workflows, and an evaluation report. The project demonstrates the feasibility and limitations of integrating LLMs into security operations while highlighting governance and operational controls required for safe adoption.

3.Introduction and Background

Cybersecurity operations produce high volumes of human-readable artifacts—alerts, analyst notes, vendor reports, threat-intelligence web pages, and PDFs—and require repeated procedural actions such as telemetry coverage assessment, targeted testing, and red-team orchestration.

Manual processing of these artifacts is time consuming and error prone. Simultaneously, LLMs have matured into powerful tools for natural language understanding and generation that can accelerate comprehension and decision-making when combined with deterministic tools.

Recent work in security automation highlights the promise of augmenting analyst workflows with intelligent assistants, but practical adoption is hampered by risk factors: hallucination, insecure execution of tooling, improper handling of credentials, and lack of reproducible audit trails.

There is a need for a principled framework that integrates LLM capabilities (summarization, instruction decomposition) with safe, auditable tool invocation to speed routine tasks (triage, telemetry analysis) and to enable programmatic interaction with emulation/testing frameworks (e.g., CALDERA) in contained environments.

This project builds on that need by designing agent templates and tool wrappers that expose common security operations to LLMs while enforcing operational controls (sandboxing, truncation, approvals) and producing reproducible notebooks and scripts for evaluation and demonstration.

4. Problem Statement

- Security teams need an integrated, auditable system that enables natural-language-driven automation to:
- Ingest heterogeneous textual sources (web pages, PDFs, telemetry exports) and produce concise, accurate summaries and structured findings.
- Identify categories of telemetry not detected by a given EDR export and present those gaps in actionable form.
- Safely execute diagnostic shell commands in a sandbox for evidence collection and supporting the LLM's decision process.
- Programmatically interact with an adversary emulation framework (CALDERA) to enumerate capabilities, execute controlled tests, and create repeatable adversary profiles.
- The requirements are precise: the system must be demonstrably reproducible, preserve operator control for sensitive actions, and provide measurable improvements in analyst throughput and diagnostic accuracy.

5. Objectives of the Project

1. Implement a set of LLM-driven agent templates that perform text analysis, internet ingestion, coordination, sandboxed command execution, and CALDERA orchestration.

2. Build deterministic tool wrappers for web scraping, PDF extraction, shell execution, and Caldera API interactions; register these tools with agents in a clearly auditable manner.

3. Implement telemetry-gap detection that ingests EDR telemetry datasets and outputs categorized detection deficits with an evaluation against a labeled baseline.

4. Demonstrate end-to-end workflows using Jupyter notebooks and orchestration scripts that validate safety controls (sandboxing, credential separation, operator approval).

5. Evaluate system performance across accuracy (summarization, telemetry-gap detection), efficacy (time-to-result vs. manual), and safety (incidence of unsafe actions prevented by controls).

6. Proposed Methodology

Agent Layer (LLM-driven):

task_coordinator_agent: breaks complex tasks into steps, selects agents/tools, sequences execution.
text_analyst_agent: summarizes and extracts structured information from texts (reports, web pages, PDFs).
internet_agent: fetches remote content and preprocesses it for analysis.
cmd_exec_agent: requests sandboxed shell command execution and interprets outputs.
caldera_agent: formulates API requests to CALDERA and interprets results.
Tool Layer (deterministic, auditable functions):

web_tools:
download_web_page (BeautifulSoup)
download_pdf_report (pypdf)
detect_telemetry_gaps (curl + jq parsing).

- **code_tools:**
  exec_shell_command (runs in a defined CODE_WORKING_FOLDER sandbox).

caldera_tools: caldera_api_request, caldera_api_method_details, caldera_swagger_info, caldera_execute_command_on_agent, etc.

Orchestration & Interfaces:

run_agents.py / run_servers.py: runtime entry points.

Jupyter notebooks: interactive demos and reproducible scenarios.

Configuration: environment variables /. env
(LLM_WORKING_FOLDER, API keys, endpoints).

6.2 Workflow example (Telemetry-gap detection)

Analyst requests: "Assess telemetry coverage for EDR X from this JSON export."
internet_agent or text_analyst_agent downloads and parses the telemetry JSON (tools/web_tools.detect_telemetry_gaps).
task_coordinator_agent decomposes the task (parse, classify, map to ATT&CK categories).
LLM synthesizes a human-friendly report listing categories not detected and recommendations (sensor tuning, new telemetry collection).
Outputs and the raw tool outputs are stored for audit.

6.3 CALDERA orchestration workflow

1. caldera_agent queries swagger to discover API surface (caldera_swagger_info).
2. The agent requests enumeration (caldera_service_list) and executes commands on specific agents (caldera_execute_command_on_agent) if in an approved lab.
3. The agent can create adversary profiles programmatically (caldera_create_adversary_profile) based on mapped techniques.

6.4 Algorithms and models

LLM backbone: configurable via OPENAI_MODEL_NAME (e.g., gpt-3.5-turbo). Agents rely on few-shot or instruction-based prompting and tool invocation patterns supplied by an autogen ConversableAgent abstraction.
Pre/post-processing: HTML stripping (BeautifulSoup), PDF text extraction (pypdf), JSON filtering (jq invoked via subprocess).
Token management: use of tiktoken and truncate_str_to_tokens to limit prompt and response sizes.
Telemetry-gap logic: deterministic filter and mapping to categories (jq + LLM for classification/mapping).

6.5 Safety, governance and operational controls

Sandboxing: exec_shell_command runs only in CODE_WORKING_FOLDER. production deployment inside containers/VMs recommended.

Approval gates: destructive actions require explicit operator confirmation before execution; all tool invocations are logged.
Secrets: API keys and credentials stored in environment variables and never committed to source control (.env_template shows required keys).

Auditing: maintain logs of agent prompts, tool inputs/outputs, timestamps, and operator approvals.
Rate and cost control: monitor LLM usage and limit model calls in production.

6.6 Evaluation plan

Summarization accuracy: evaluate LLM summaries against human-annotated gold standards using ROUGE/BERTScore and human qualitative assessment.

Telemetry-gap detection: measure precision, recall, and F1 against labelled telemetry coverage datasets.

Operational effectiveness: time-to-complete triage tasks vs. control group (manual triage).

Safety checks: count prevented unsafe actions due to approval gate and sandbox tests.

6.7 Experimental setup

Environment: Docker containers with reproducible dependencies; use of a test Caldera server and synthetic EDR telemetry exports.
Datasets: curated set of threat reports, web pages, and telemetry JSON exports (anonymized).
Metrics and logging: structured logs stored centrally for offline analysis.

7. Technologies / Tools to be Used

1. **Programming language**: Python 3.9+
2. **Interactive:** Jupyter Notebook
3. **LLM integration:** autogen ConversableAgent abstraction; OpenAI API (configurable via OPENAI_MODEL_NAME / OPENAI_API_KEY); tiktoken for token accounting
4. **Web & parsing:** BeautifulSoup (bs4), pypdf
5. **Networking & HTTP**: requests, curl (used via subprocess)
6. **Shell & JSON tools:** subprocess, jq (shell utility)
7. **Red-team framework:** MITRE CALDERA (API integration)
8. **Containerization & sandboxing:** Docker (recommended)
9. **Version control:** Git / GitHub
10. **Utilities**: typing_extensions, logging, standard Python libraries
11. **Deployment & orchestration scripts:** run_agents.py, run_servers.py, run_notebooks.sh
12. **Configuration:** env style configuration (example. env_template includes LLM_WORKING_FOLDER, CALDERA_SERVER, CALDERA_API_KEY,OPENAI_MODEL_NAME, OPENAI_API_KEY, optional web/FTP parameters).
13. **Optional:** CI/CD GitHub Actions (the repository contains an actions directory).

8. Expected Outcomes and Applications

- Implemented, documented codebase with agent templates and tool wrappers.
- Jupyter notebooks demonstrating:
    - Text and web ingestion with summarization.
    - Telemetry-gap detection from EDR exports.
    - Safe sandboxed shell command execution and output capture.
    - CALDERA interactions: listing services, creating adversary profiles, exec uting non-destructive commands.
- Evaluation report with quantitative and qualitative analysis of accuracy, time savi ngs, and safety metrics.
- Setup and operational documentation, including guidance on sandboxing, credenti al management, and approval workflows.

8.2 Measurable results

- Summarization: target ROUGE/BERTScore benchmarks and a human evaluation i ndicating improved analyst throughput.
- Telemetry detection: a target F1 score to demonstrate utility against baseline manu al inspection.
- Productivity: measurable reduction in average time to triage/produce a telemetry r eport in demo scenarios.

8.3 Applications and target users

- SOC analysts: accelerate alert triage and produce standardized summaries of incid ents and reports.
- Threat intelligence analysts: ingest and summarize web intelligence and PDF repo rts automatically.
- Red-team and purple-
  team engineers: programmatically create repeatable adversary profiles and orchest rate emulation scenarios.
- Security automation engineers: prototype LLM-
  augmented automation playbooks and integrate them into broader security automa tion pipelines.

9. Conclusion

This project demonstrates a practical, modular approach to augmenting cybersecurity workflows with LLM-driven conversational agents and deterministic tool wrappers. The implemented prototype—comprised of agent templates (coordinator, text/internet, command-execution, Caldera integration), tool functions (web/PDF ingestion, telemetry-gap detection, shell execution, Caldera API helpers), and reproducible notebooks—shows that LLMs can meaningfully accelerate analyst tasks such as document triage, telemetry assessment, and adversary-emulation orchestration. At the same time, the work highlights important operational, security, and governance constraints that must be addressed before production use.

Key conclusions

Feasibility: LLMs are effective at decomposing natural-language objectives, summarizing heterogeneous text sources, and producing actionable recommendations when combined with concrete tool outputs. The Conversable Agent + tool pattern provides a clear, extensible architecture for bridging language understanding with deterministic operations.
Utility: For SOC analysts, threat intel teams, and red/purple teams, the repository provides practical building blocks to shorten time-to-insight (document summarization, telemetry-gap reports) and to automate repeatable emulation tasks (Caldera orchestration).
Limitations: LLM-produced outputs can be imprecise or hallucinated; tools that execute commands or access networks introduce real risk; and token/context limits constrain how much raw data can be provided to the model. Additionally, the repo is a prototype (demo notebooks, no formal license, limited audit/logging, no RBAC or secrets-management).
Safety and governance are non-negotiable: any deployment that executes shell commands, manipulates emulation infrastructure, or transmits data must include sandboxing, operator approvals, detailed audit trails, secret isolation, and rate/cost controls.
Operational recommendations before real-world adoption

Access control & approvals:
Implement an explicit approval workflow that requires human confirmation for any destructive or network-changing action (e.g., file upload, command execution, Caldera agent commands).
Secrets & credentials:
Use secure secret stores (Vault, AWS Secrets Manager) rather than plaintext .env files in production. Rotate API keys regularly.
Auditing & logging:
Log all prompts, tool inputs/outputs, operator decisions, timestamps, and contextual metadata for forensic review and model governance.
Validation & human-in-the-loop:
Mandate human verification of critical outputs (e.g., telemetry-gap remediation actions, adversary profile creation). Use LLM outputs as assistive guidance not autonomous operators.
Cost and usage governance:
Monitor model usage, set quotas, and choose model variants consistent with latency, accuracy, and cost constraints.

Accuracy: Measure summarization quality (ROUGE/BERTScore) and telemetry-gap detection precision/recall against labeled datasets.
Efficiency: Measure time-to-triage and time-to-report compared to manual baselines.
Safety: Track prevented unsafe actions via approval gates and count any sandbox escapes or unapproved remote interactions during testing.
Usability: Collect analyst satisfaction and perceived usefulness using structured surveys after pilot runs.

This project is a valuable, well-constructed prototype that provides concrete, reusable patterns for integrating LLMs into security workflows. It is best seen as a research/prototyping asset and a teaching/demo resource rather than a production-ready platform. With disciplined application of sandboxing, access controls, auditing, and human-in-the-loop governance, the architecture can be matured into a safe, effective augmentation layer for security teams—yielding measurable productivity gains while containing operational risk.

10.References

1. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language Models are Few-Shot Learners. arXiv:2005.14165. https://arxiv.org/abs/2005.14165
2. MITRE ATT&CK® Framework. https://attack.mitre.org/ — taxonomy and technique mappings for adversary behavior.
3. MITRE CALDERA. https://github.com/mitre/caldera — automated adversary emulation platform and API reference.
4. OpenAI API Documentation. https://platform.openai.com/docs/ — guidance on model usage and best practices.
5. NIST Special Publication 800-61 Revision 2, Computer Security Incident Handling Guide. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf